

# Gestione della memoria

Antonio Pecchia

antonio.pecchia@unisannio.it

Dipartimento di Ingegneria  
Laurea in Ingegneria Informatica  
Sistemi Operativi

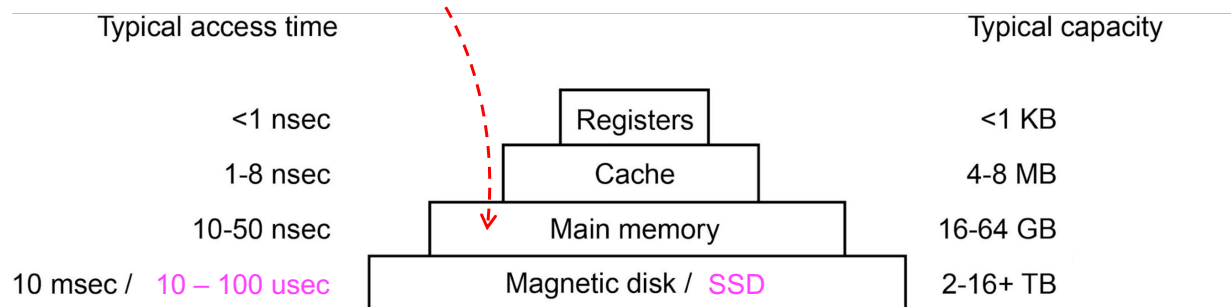


UNIVERSITÀ DEGLI STUDI  
DEL SANNIO Benevento

## Gerarchia di memoria

**main memory:** allocazione della memoria ai processi, deallocazione, protezione, condivisione controllata, indirizzi, ...

|           |             |       |      |
|-----------|-------------|-------|------|
| $10^{-3}$ | 0.001       | milli | msec |
| $10^{-6}$ | 0.000001    | micro | usec |
| $10^{-9}$ | 0.000000001 | nano  | nsec |



# NESSUNA ASTRAZIONE DI MEMORIA

Cap 3: par. 3.1

## Lo scenario

- **Nessuna** astrazione:
  - i programmi "vedono" la **memoria fisica**.

```
JMP 28  
MOV R1,1000
```

memoria fisica

|        |       |
|--------|-------|
|        | 16380 |
| ⋮      |       |
| CMP    | 28    |
|        | 24    |
|        | 20    |
|        | 16    |
|        | 12    |
|        | 8     |
|        | 4     |
| JMP 28 | 0     |

## Codice assoluto (absolute code)

- L'associazione di **istruzioni/dati** ad **indirizzi** di memoria è fatta a tempo di **compilazione**:

- in fase di compilazione è noto dove il processo risiederà in memoria;
- se, in un momento successivo, la locazione cambiasse, sarebbe necessario ricompilare il codice.

memoria fisica

|        |       |
|--------|-------|
|        | 16380 |
| ⋮      |       |
| CMP    | 28    |
|        | 24    |
|        | 20    |
|        | 16    |
|        | 12    |
|        | 8     |
|        | 4     |
| JMP 28 | 0     |

## Caso monoprogrammato

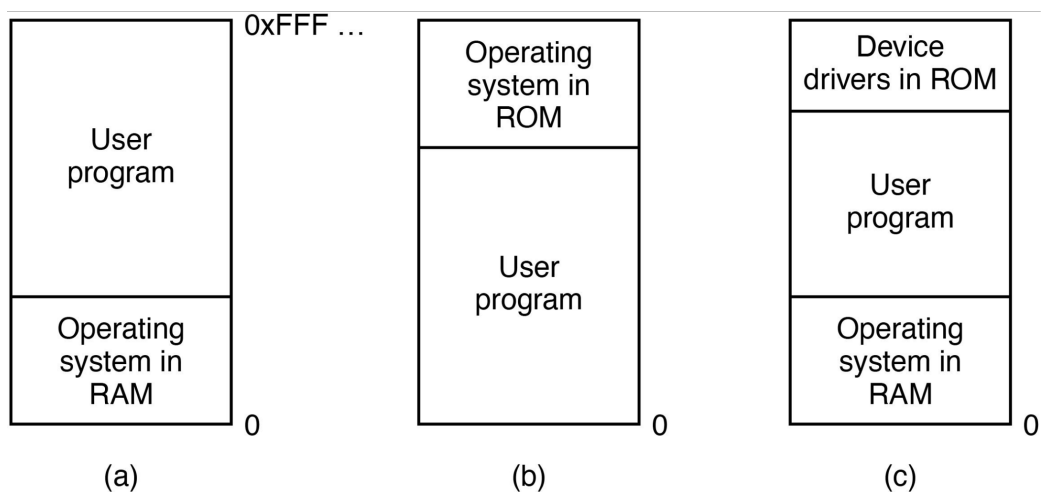
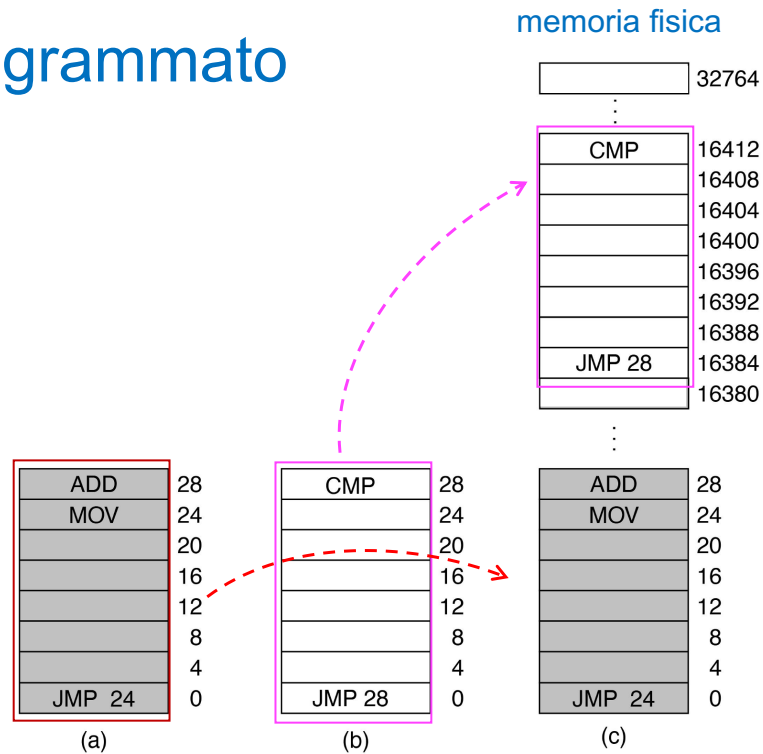


Figure 3-1. Three simple ways of organizing memory with an operating system and **one user process**.

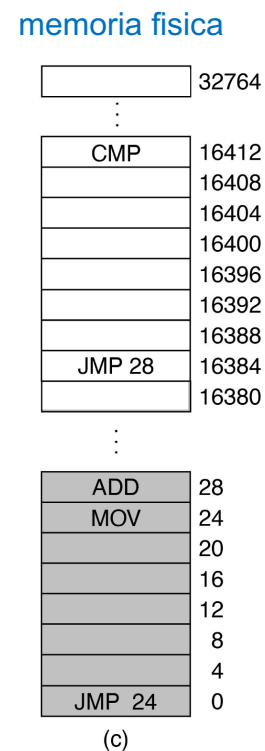
## Caso multiprogrammato



## Caso multiprogrammato

### □ Possibile soluzione:

- rilocalizzazione statica;
- protezione della memoria;

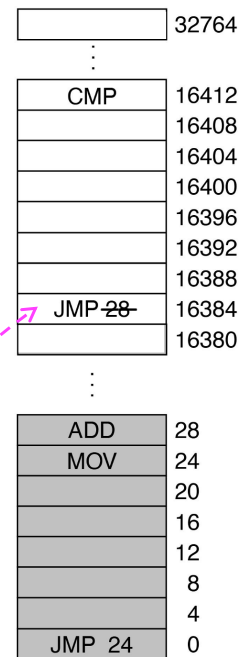


## Rilocazione statica

- Modifica del programma, "al volo", quando è **caricato** in memoria:
  - noto l'indirizzo di caricamento (16384 in figura), viene aggiunta la costante 16384 ad ogni indirizzo del programma;
  - il compilatore deve generare codice *rilocabile*.

JMP 28 alla locazione 16384 diventa JMP 16412

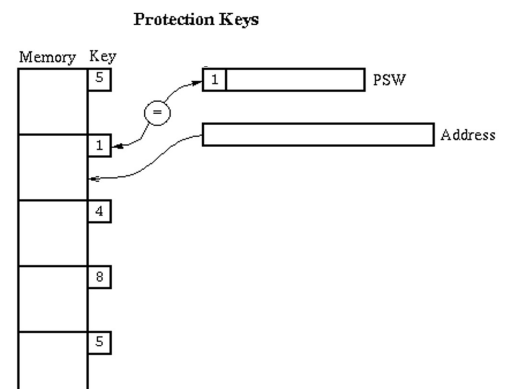
memoria fisica



(c)

## Protezione memoria

- La memoria è suddivisa in blocchi, e a ciascuno è assegnata una **chiave di protezione**.
- Il PSW contiene una chiave;
- Il SO intercetta i tentativi di accedere a blocchi di memoria con una chiave diversa da quella presente nel PSW.

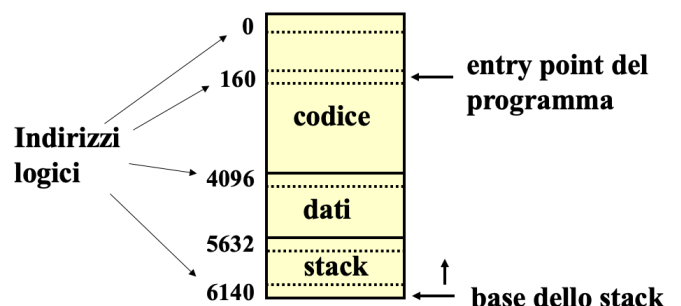


# SPAZIO DEGLI INDIRIZZI

Cap 3: par. 3.2 (3.2.1 e 3.2.2)

## Spazio degli indirizzi

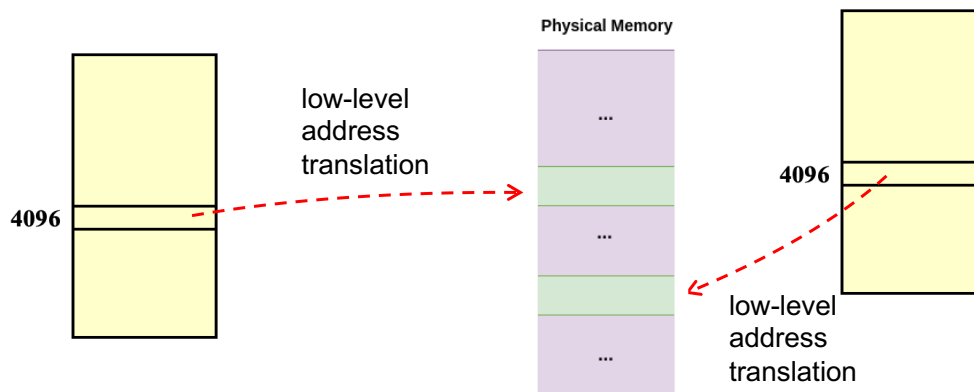
- Insieme degli indirizzi che un processo può usare per indirizzare la memoria:
  - ogni processo ha il suo spazio degli indirizzi "personale", **indipendente** dagli altri processi;
  - **problema: come dare** ad ogni processo il proprio spazio degli indirizzi?



PC, SP contengono indirizzi logici, ptr a variabili sono indirizzi logici, istruzioni "tipo" jmp riferiscono indirizzi logici, ...

## Spazio degli indirizzi

- L'indirizzo **4096** in un programma "significa" una locazione fisica diversa dall'indirizzo **4096** di un altro programma.



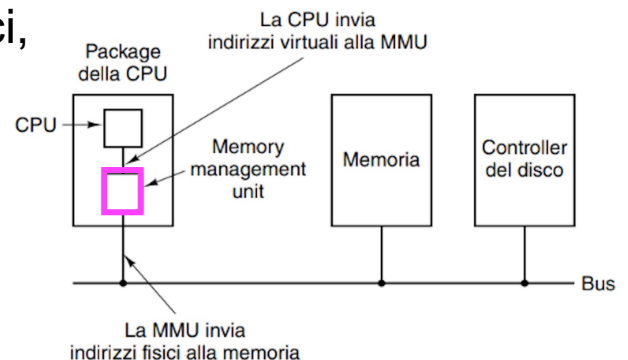
## Rilocazione dinamica

- L'associazione di **istruzioni/dati** ad **indirizzi** di memoria è fatta a tempo di **esecuzione**:
  - gli indirizzi logici **non coincidono**\* con gli indirizzi fisici;
  - richiede il supporto specifico dell'architettura (per es., specifici **registri hardware**);
  - durante il suo ciclo di vita un processo può essere spostato da un blocco di memoria ad un altro.

\*NOTA: come detto in precedenza, nel metodo di associazione in fase di **compilazione**, gli indirizzi logici e fisici sono identici; nell'associazione in fase di **caricamento** gli indirizzi logici (rilocabili) sono rimpiazzati da quelli fisici.

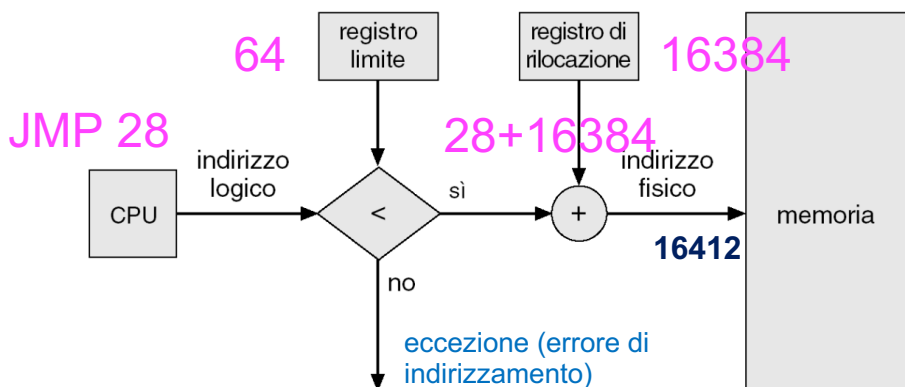
# Memory management unit (MMU)

- Indirizzo **logico** (*logical* address): indirizzo generato dalla CPU, detto anche indirizzo virtuale (*virtual* address);
- Indirizzo **fisico** (*physical* address): indirizzo visto dall'unità di memoria;
- Il programma tratta indirizzi logici, *non* indirizzi fisici:
  - MMU**: dispositivo HW per l'associazione degli indirizzi logici agli indirizzi fisici in fase di esecuzione.



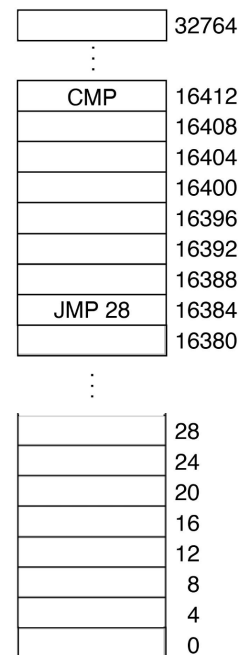
## Un primo esempio

- Registro **limite**: limite superiore dello spazio logico del programma.
- Registro **base\***: indirizzo fisico di inizio.



\*registro base o anche registro di rilocalione

### memoria fisica

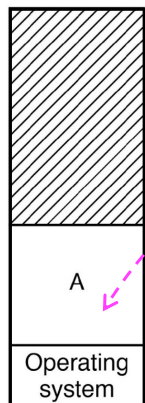


(b)

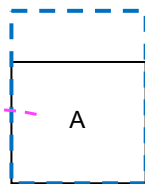
(c)



memoria fisica

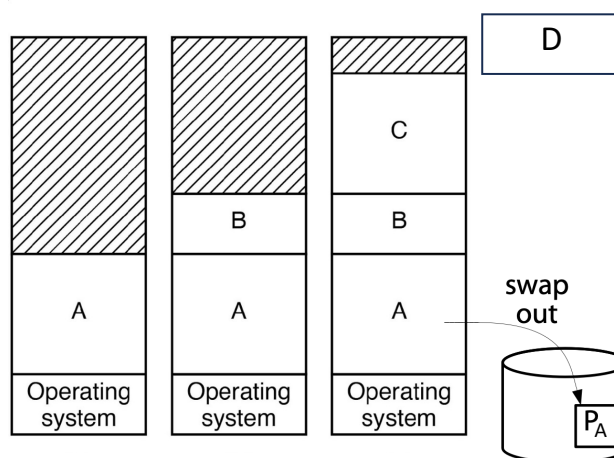


indirizzi logici su 16 bit (0—65535)  
 $2^{16} = 2^6 \times 2^{10} = 64 \text{ KB}$



(usato da A = 48 KB)

Figure 3-4. Memory allocation changes as processes come into memory and leave it.



\* il processo, nella sua totalità, è posto  
nella memoria *non volatile*.

Figure 3-4. Memory allocation changes as processes come into memory and leave it.

# Swapping

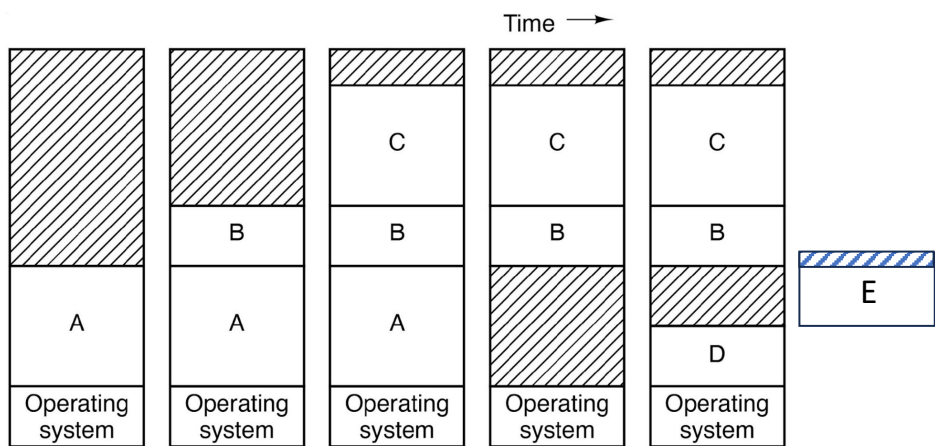


Figure 3-4. Memory allocation changes as processes come into memory and leave it.

# Swapping

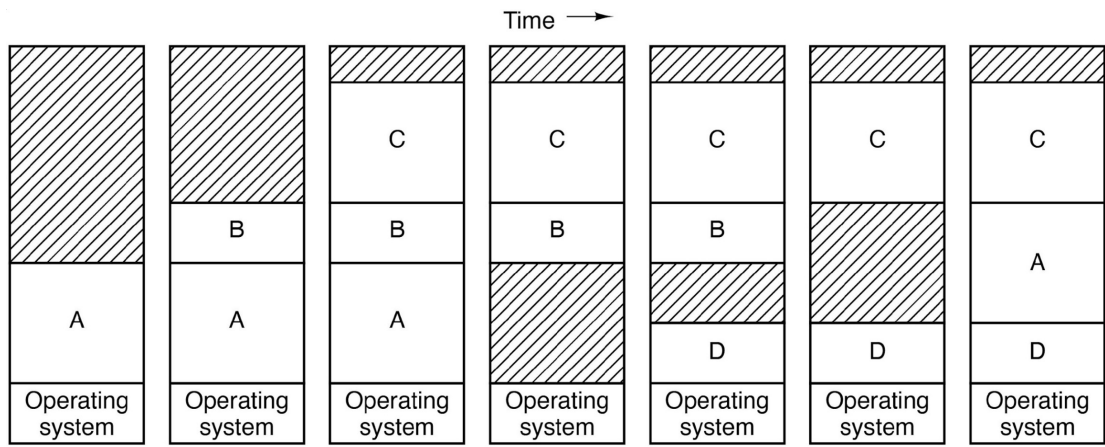
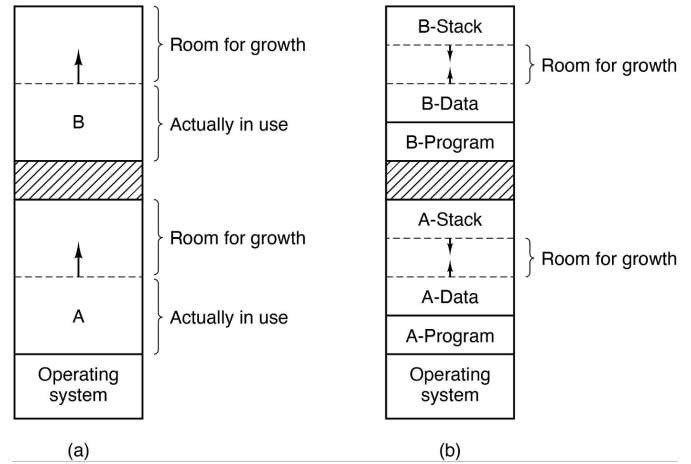


Figure 3-4. Memory allocation changes as processes come into memory and leave it.

## Ulteriori aspetti

- Bisogna garantire **spazio "extra"** poiché i processi possono crescere (per es., area heap);
- Problematiche:
  - quanto spazio extra?
  - come procedere in caso di out-of-memory (OOM)?
    - "uccidere" il processo;
    - rilocarlo;
    - effettuare uno swap out;



# MEMORIA VIRTUALE

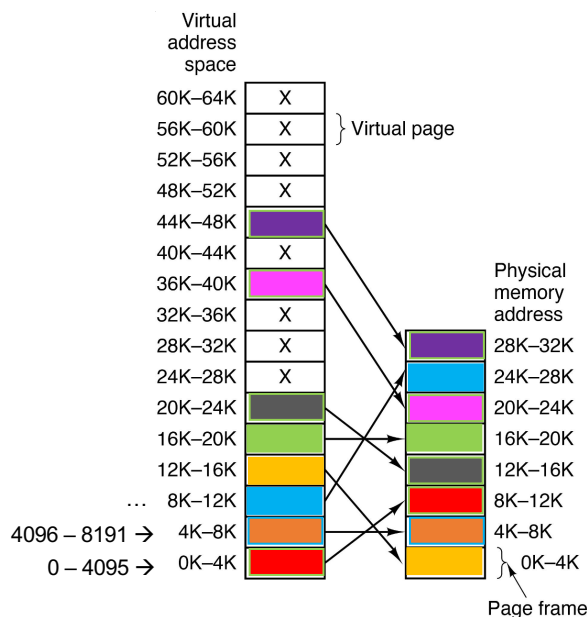
Cap 3: par. 3.3 – 3.3.1, 3.3.2, 3.3.3 e 3.3.4 (escluso "Tabelle invertite delle pagine")

## Problemi e soluzione

- ❑ Per quanto visto fino ad ora, un processo deve essere sempre collocato in una porzione di memoria **contigua**;
- ❑ Si pone anche il problema di come eseguire programmi *più grandi* della memoria fisica disponibile:
  - problema risalente già agli anni '60 (soluzione: **overlay**);
- ❑ **Soluzione: memoria virtuale**. Idea di base:
  - un programma ha il suo spazio degli indirizzi **suddiviso** in pagine\* (schema tipico: **paginazione** o *paging* con unità di dimensione fissa pari a 4KB).

\*Altra possibilità: suddivisione in **segmenti** di dimensione variabile (**segmentazione**).

# Paginazione



indirizzi virtuali su 16 bit (0—65535)  
 $2^{16} = 2^6 \times 2^{10} = 64 \text{ KB}$

Dimensione pagina: 4 KB  $\rightarrow$  16 pagine

Dimensione memoria fisica: 32 KB

Dimensione pagina == dimensione frame

x  $\rightarrow$  pagina virtuale *non mappata* in memoria fisica

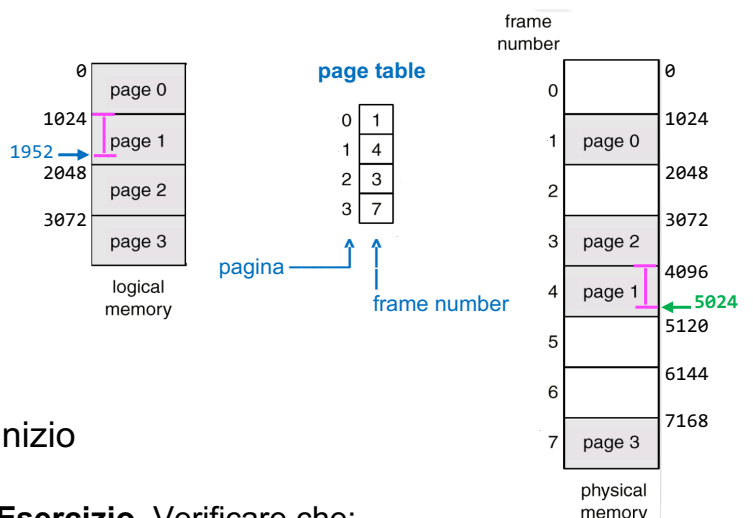
\*In generale, se dim. memoria =  $2^m$  e dim. pagina =  $2^n$ , il numero totale di pagine è  $2^{(m-n)}$

## Esempio

### Indirizzo logico 1952:

- appartiene a **page 1**, mappata su **frame 4** (che inizia a 4096);
- offset** dell'indirizzo logico rispetto all'inizio della pagina di appartenenza (**page 1**):
  - $1952 - 1024 = 928$
- indirizzo fisico**: indirizzo fisico di inizio frame + **offset**:
  - $4096 + 928 = 5024$

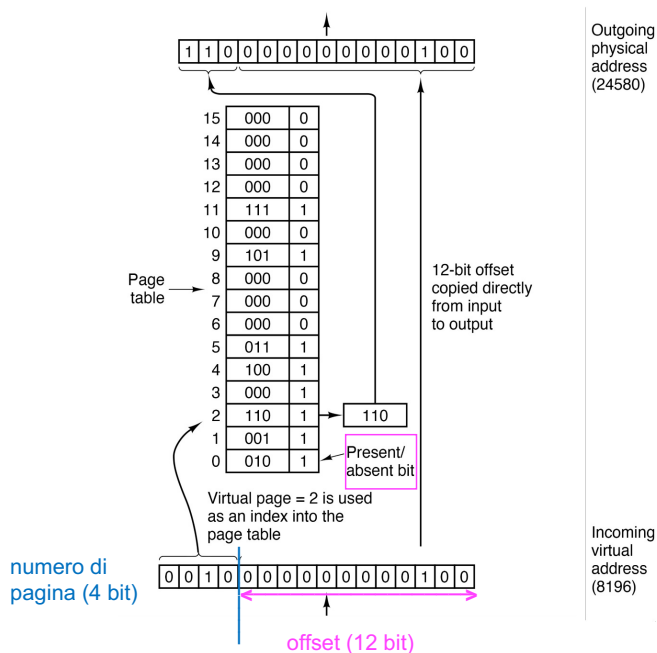
pagina = 1K ; memoria = 8K



**Esercizio.** Verificare che:

- ind. logico 936 diventa ind. fisico 1960
- ind. logico 2049 diventa ind. fisico 3073

# Traduzione e tabella delle pagine



Con indirizzi virtuali su 16 bit (0—65535) e dimensione pagina 4 KB (ossia  $2^{12}$ ):

$$2^{16}/2^{12} = 2^{16-12} = 2^4 \rightarrow 16 \text{ pagine}^* (0—15)$$

Nell'esempio, la memoria fisica è 32 KB (ossia  $2^{15}$ ), il numero totale di frame è:

$$2^{15}/2^{12} = 2^{15-12} = 2^3 \rightarrow 8 \text{ frame} (0—7)$$

**bit present/absent\*\***: la pagina è in memoria (1) oppure la pagina è su disco o non mappata (0); se il bit è 0, la CPU genera un **page fault**.

\*In generale, se dim. memoria =  $2^m$  e dim. pagina =  $2^n$ , il numero totale di pagine è  $2^{(m-n)}$

\*\*anche noto come **valid/invalid** bit.

## Un po' di numeri...

- Quante pagine ottengo nel caso di indirizzi a 64 bit e dimensione di pagina di 4 KB?

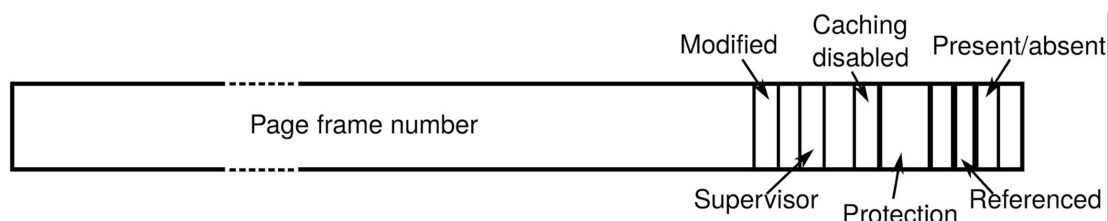
$$2^{64} / 2^{12} = 2^{(64-12)} = 2^{52}$$

- in pratica, nelle CPU a 64 bit sono usati 48 bit dell'indirizzo virtuale:
  - i 16 bit più significativi sono settati a 0 per lo spazio degli indirizzi virtuale utente e 1 per quello kernel;
  - numero di pagine:

$$2^{48} / 2^{12} = 2^{(48-12)} = 2^{36}$$

## Struttura di una page table entry

- Dettaglio di una singola voce (**entry**) della tabella delle pagine.
  - dimensione **totale** della entry: **64 bit**
    - **52 bit** più significativi → numero del frame;
    - **12 bit** restanti → informazioni sulla pagina.



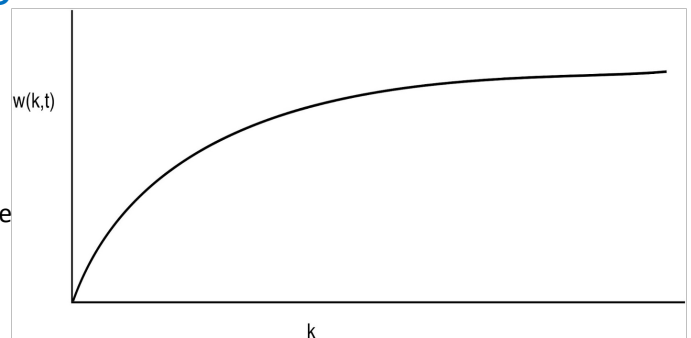
## Paging: implementazione

- Aspetti da considerare nell'implementazione:
  - la traduzione indirizzo virtuale – fisico **deve essere veloce**;
  - la tabella delle pagine **non deve essere** troppo "grande";
  - *reminder*: ogni processo ha la sua tabella delle pagine!

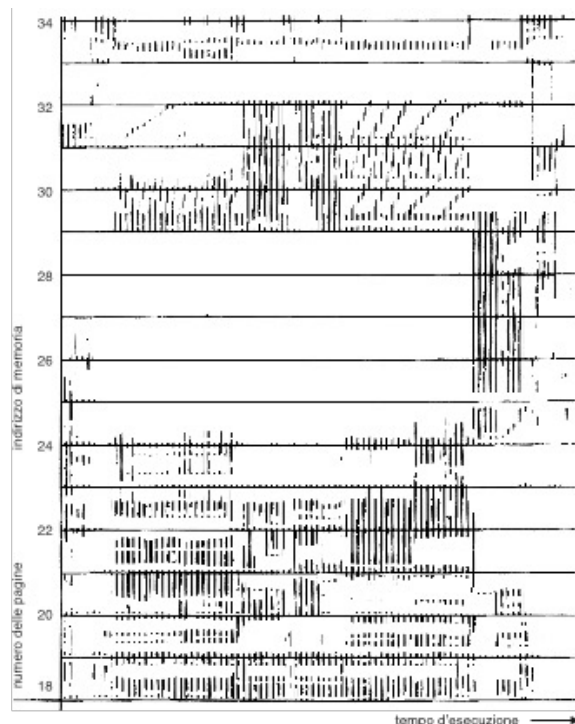
## Alcune considerazioni

- ❑ **Località di riferimento**: un processo fa riferimento solo a una frazione relativamente piccola delle sue pagine;
- ❑ **Working set**: insieme delle pagine che un processo sta usando "attualmente";
- ❑ **Demand paging vs pre-paging.**

Figure 3-18. The working set is the set of pages used by the  $k$  most recent memory references. The function  $w(k, t)$  is the size of the working set at time  $t$ .



## Alcune considerazioni





## Paging: opzioni implementative

- ❑ Tabella delle pagine *interamente* come **registri HW**:
  - **pro**: efficienza | **contro**: necessiterebbe di troppi registri HW + forte overhead dovuto ai context switch;
- ❑ tabella delle pagine *interamente* **in memoria** più (almeno) un registro HW detto PTBR (page table base register)\*:
  - **pro**: 1-2 registri HW + context switch veloce | **contro**: inefficienza;
- ❑ **in pratica**: tabella delle pagine in **memoria**, PTBR e **TLB**.

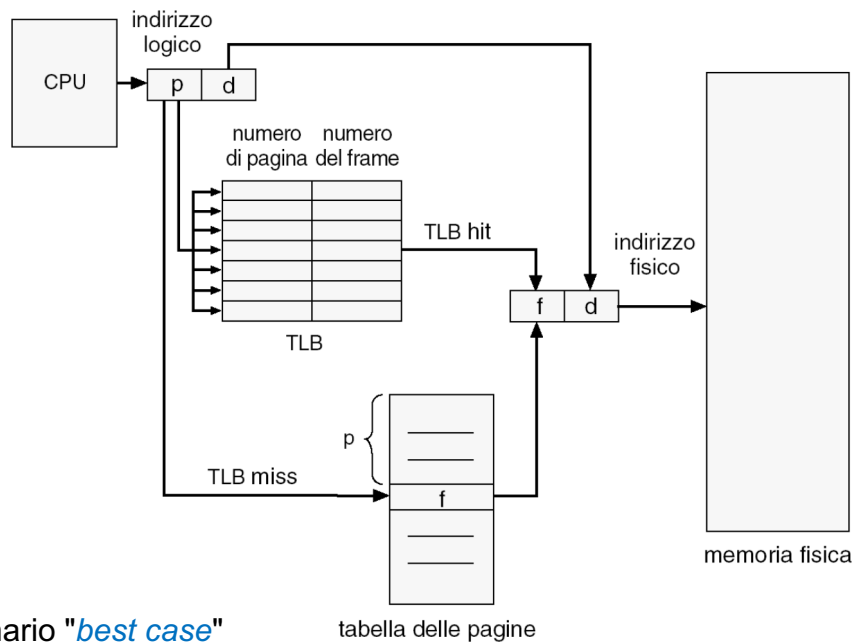
\* il registro PTBR punta all'*inizio* della tabella delle pagine (per es., negli x86 il registro CR3 funge da *PTBR*); opzionalmente può esserci anche un *page-table length register* (PTLR) contenente dimensione della tabella delle pagine.

## Translation lookaside buffer (TLB)

- ❑ TLB: **dispositivo HW** (cache associativa) della MMU per mappare gli indirizzi virtuali *senza passare* dalla tabella delle pagine:
  - numero ridotto di entry (per es., 256).

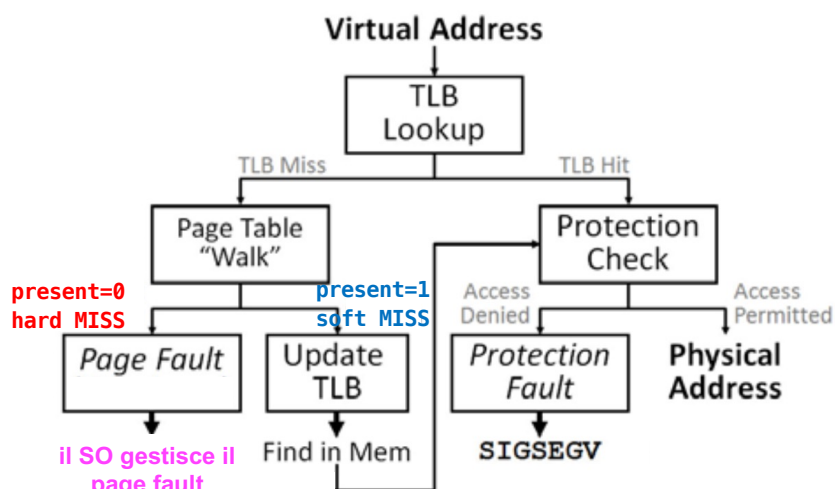
| Valid | Virtual Page | Modified | Protection | Page Frame |
|-------|--------------|----------|------------|------------|
| 1     | 140          | 1        | RW         | 31         |
| 1     | 20           | 0        | R X        | 38         |
| 1     | 130          | 1        | RW         | 29         |
| 1     | 129          | 1        | RW         | 62         |
| 1     | 19           | 0        | R X        | 50         |
| 1     | 21           | 0        | R X        | 45         |
| 1     | 860          | 1        | RW         | 14         |
| 1     | 861          | 1        | RW         | 75         |

## Traduzione indirizzi\*



\* NOTA: scenario "*best case*"

## Gestione TLB e page faults



\* NOTA: *workflow* a carattere illustrativo

## TLB e page faults (1/2)

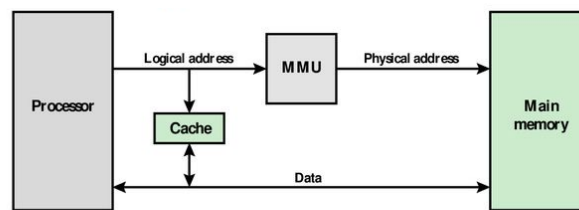
- Tipicamente, il TLB viene invalidato/svuotato nel caso di context switch da un processo all'altro\*;
- **TLB hit**: il numero di pagina virtuale è nel TLB;
- **TLB miss**: il numero di pagina virtuale **non è** nel TLB → *"page table walk"*:
  - "soft" TLB miss: il numero di pagina non è nel TLB, ma bit present=1 → (crea voce TLB / non serve I/O da disco o SSD);
  - "hard" TLB miss: bit present=0 → **page fault** (slide successiva).

\* **NOTA**: a meno che le voci del TLB non siano contrassegnate con il PID del processo a cui appartengono (in tal caso si parla di **tagged TLB**).

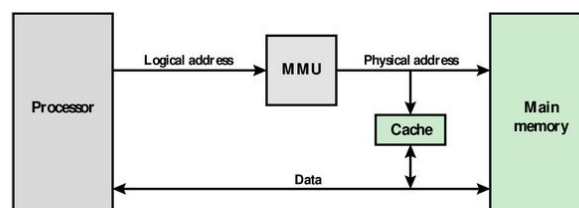
## TLB e page faults (1/2)

- Il SO gestisce i **page faults**. Possibilità:
  - **minor page fault**: la pagina è già in memoria (per es., era stata già richiesta e caricata *per conto di un altro processo*);
  - **major page fault**: la pagina **non** è in memoria (per prelevare la pagina bisognerà accedere al disco o SSD);
  - **segmentation fault**: il programma ha provato ad accedere ad un indirizzo non valido; il SO terminerà il programma.

## Sidenote: cache e MMU



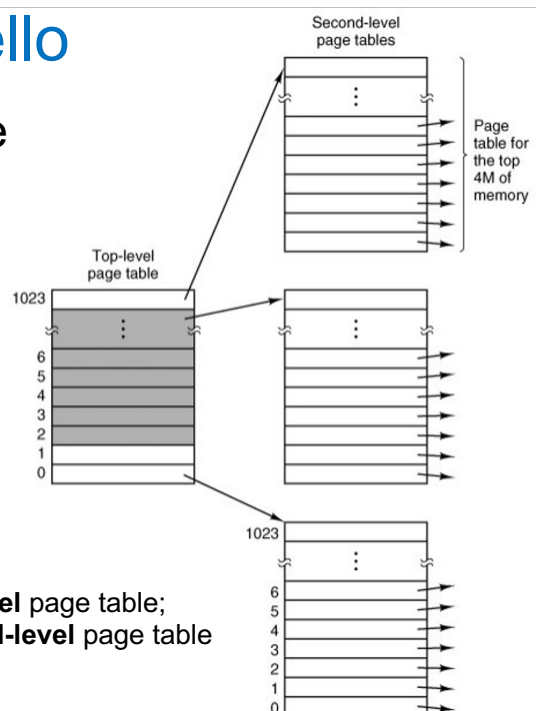
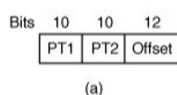
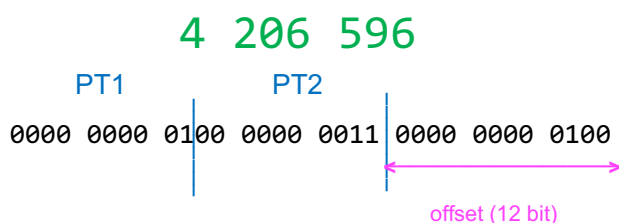
(a) Logical Cache



(b) Physical Cache

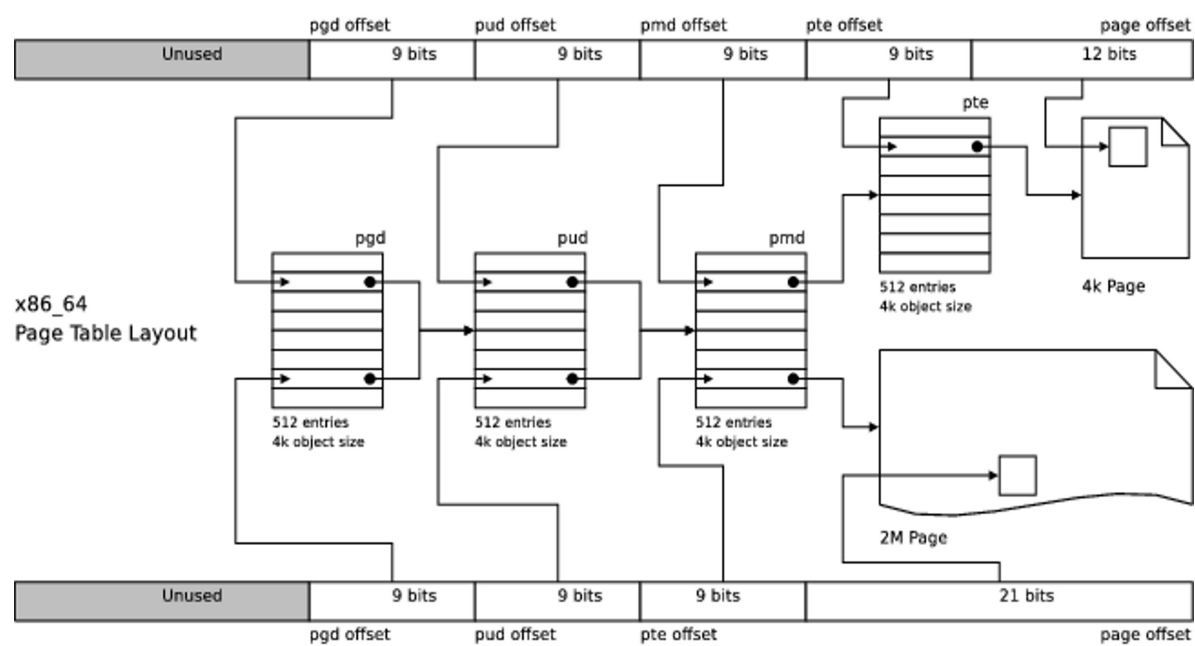
## Tabella delle pagine multilivello

- Approccio per limitare la dimensione della tabella delle pagine.
- Esempio (32 bit)- ind. virtuale:



PT1 consente di accedere alla voce 1 (nell'esempio) della **top-level** page table;  
 PT2 consente di accedere alla voce 3 (nell'esempio) della **second-level** page table  
 puntata dalla voce 1 della top-level page table;

# Four-level page table (x86-64)



# GESTIONE PAGE FAULT E SOSTITUZIONE DELLE PAGINE

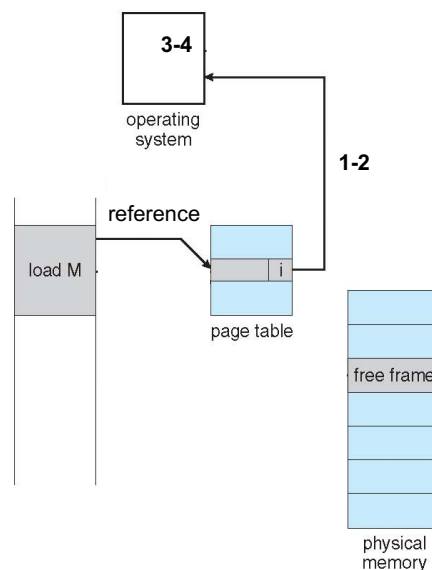
Page fault: Cap. 3: par. 3.6.2;  
Sostituzione delle pagine: Cap. 3: par. 3.4  
(escluso 3.4.6, 3.4.7, 3.4.9 e 3.4.10).

## Gestione di un page fault\*

**1-2** passi base gestione eccezioni;  
invocazione **gestore dei page fault**;

**3** il SO determina la pagina virtuale  
necessaria;

**4** il SO verifica che l'**indirizzo virtuale**  
sia valido; in caso affermativo, si  
individua **a)** un **frame libero** in memoria  
centrale (se esiste), *oppure b)* un frame  
da liberare tramite un **algoritmo di  
sostituzione delle pagine**;

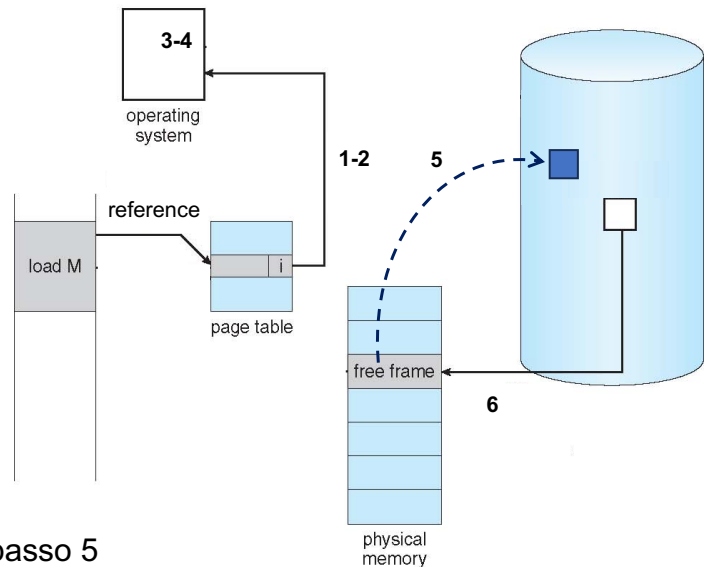


\*NOTA: numerazione utilizzata dal libro di testo par. 3.6.2.

## Gestione di un page fault

**5** se la pagina nel frame individuato al passo 4 è "sporca", viene schedulata per il trasferimento in memoria non volatile;

**6** la pagina richiesta viene caricata nel frame individuato al passo 4.



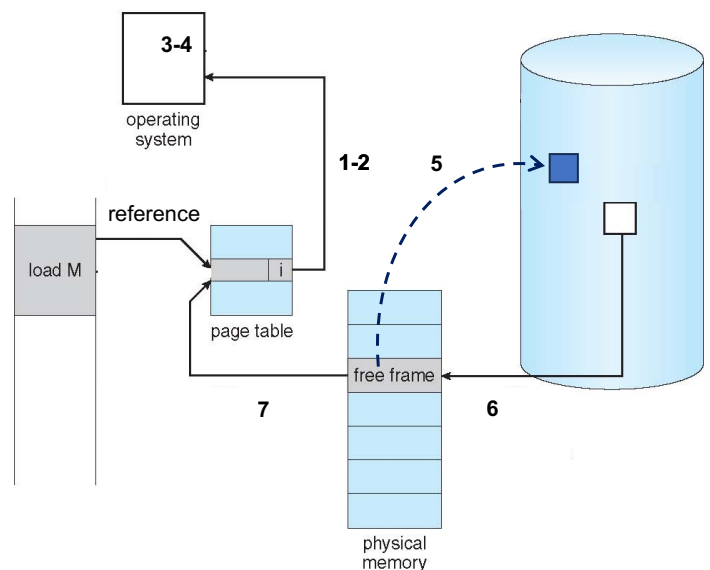
\*NOTA: il processo in page fault è sospeso al passo 5 (al passo 6, esso è ancora sospeso).

## Gestione di un page fault

**7** l'interrupt del disco segnala l'arrivo della pagina e **la tabella delle pagine viene aggiornata**;

**8** ripristino istruzione originariamente in errore e del valore del PC;

**9-10** il processo "eventualmente" verrà schedulato per riprendere l'esecuzione da dove si era interrotta.



\*NOTA: passi da 8-10 non rappresentati

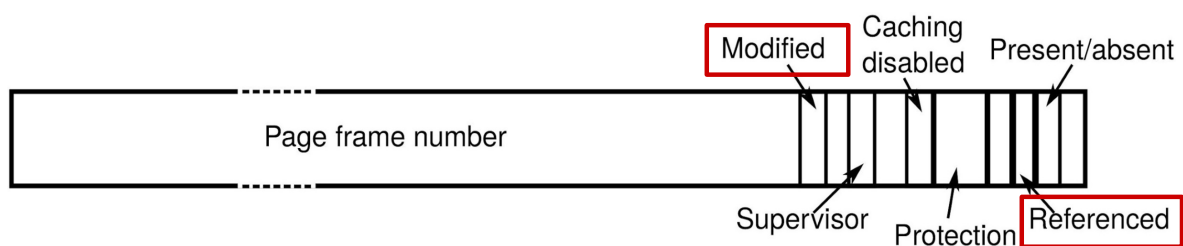
## Algoritmi di sostituzione delle pagine

- All'occorrenza di un **page fault** *potrebbe* essere necessario scegliere una pagina da **rimuovere** dalla memoria centrale. Alcuni aspetti da considerare:
  - la pagina "candidata" all'eliminazione potrebbe essere stata modificata;
  - quale pagina rimuovere?
  - si seleziona tra le pagine del processo in errore, o può essere una pagina di un altro processo?

## Supporto agli algoritmi di sostituzione

Bit rilevanti disponibili in una PTE:

- **modified (M)**: *settato* quando la pagina viene modificata (noto anche come **"dirty" bit**);
- **referenced (R)**: *settato* quando si fa riferimento alla pagina (noto anche come **"accessed" bit**).





## Algoritmo di sostituzione ottimale

- ❑ Ogni pagina è *etichettata* con il numero di istruzioni da eseguire prima che essa riceva un riferimento;
- ❑ L'algoritmo rimuove la pagina con l'etichetta massima.

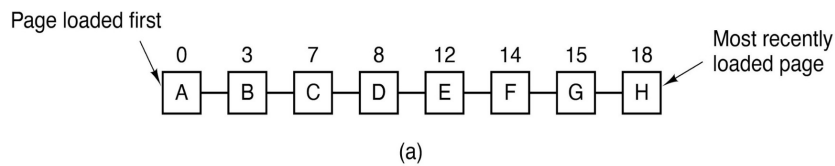
algoritmo non realizzabile in pratica

## Not recently used (NRU)

- ❑ I bit M ed R sono impostati a 0 all'avvio di un processo:
  - il bit R è azzerato **periodicamente** (per es., ad ogni *clock interrupt*);
  - durante l'esecuzione ne conseguono 4 classi di pagine:
    - Class 0: R=0, M=0
    - Class 1: R=0, M=1
    - Class 2: R=1, M=0
    - Class 3: R=1, M=1
- ❑ l'algoritmo NRU rimuove una pagina a caso dalla classe **non vuota** con numero più "basso".

## First-in, First-out (FIFO)

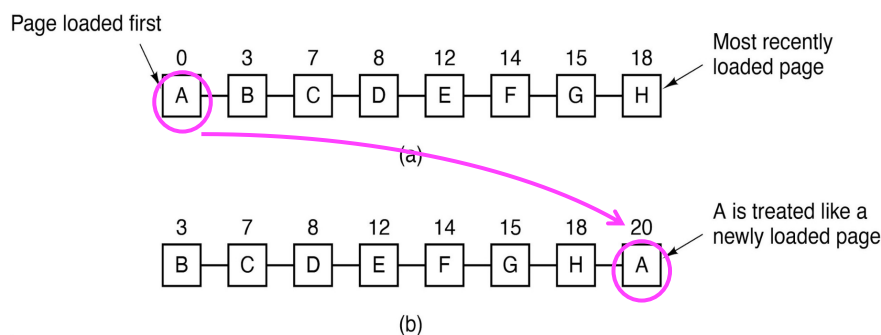
- Le pagine presenti in memoria sono organizzate in una coda, di cui la più "vecchia", in testa; la più recente, in coda.



Ad un page fault, la pagina di testa viene **rimossa**, mentre la nuova è inserita in coda.

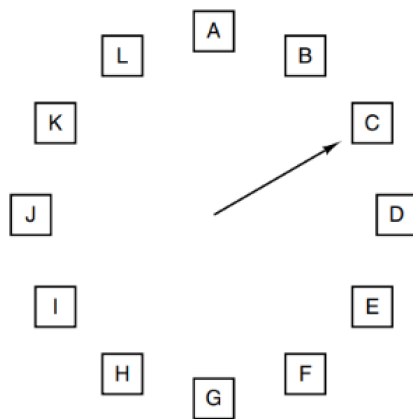
## Second chance

- Miglioria di FIFO: **controllo del bit R** della pagina più "vecchia":
  - se  $R=0$ , la pagina è sostituita; se  $R=1$ , il bit è azzerato, la pagina è posta in testa e si aggiorna l'istante di caricamento.



## Clock

- Approccio di sostituzione basato su lista circolare a forma di orologio:
  - la **lancetta** punta alla pagina più "vecchia".



Quando si verifica un page fault, viene analizzata la pagina cui sta puntando la lancetta. L'azione intrapresa dipende dal bit R:  
R = 0: Rimuovi la pagina  
R = 1: Azzera R e fai avanzare la lancetta

## Alcune considerazioni

- **Località di riferimento**: un processo fa riferimento solo a una frazione relativamente piccola delle sue pagine;
- **Working set**: insieme delle pagine che un processo sta usando "attualmente";
- Possibili strategie:
  - **demand paging**: le pagine sono caricate *on-demand*;
  - **pre-paging** caricamento delle pagine *prima* di lasciar eseguire i processi.

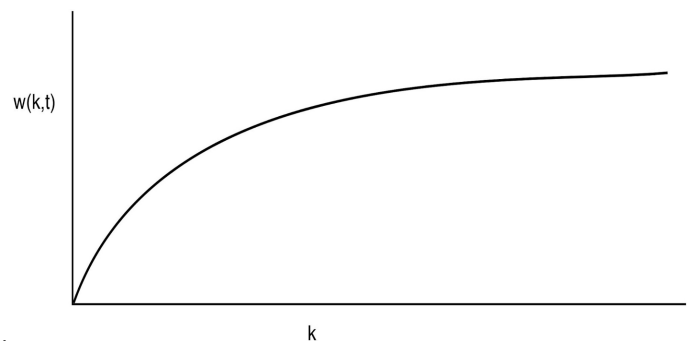
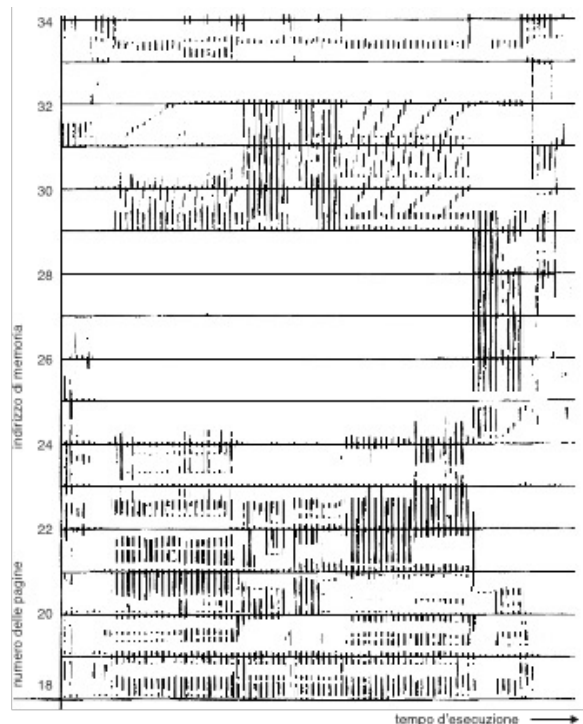


Figure 3-18. The working set is the set of pages used by the  $k$  most recent memory references. The function  $w(k, t)$  is the size of the working set at time  $t$ .

## Alcune considerazioni



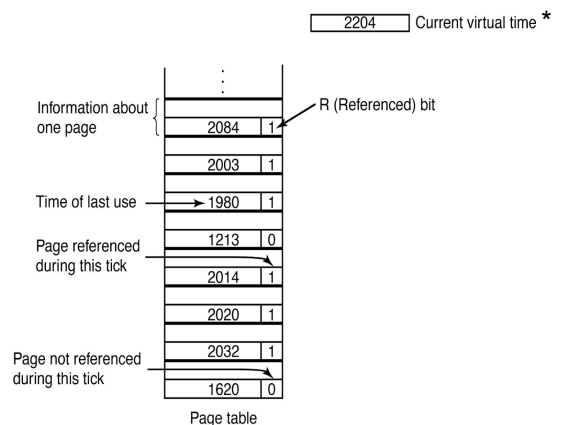
## Algoritmo working set

- **Intuizione:** in caso di page fault, si rimuove una pagina **fuori** dal working set. Nella pratica:
  - il **working set** è definito su **base temporale** (piuttosto che sul numero di riferimenti): insieme delle pagine riferite dal processo nell'ultimo lasso di tempo (virtuale\*)  $\tau$ .

### □ Assunzioni:

- nella PTE è disponibile l'**istante di ultimo utilizzo** della pagina;
- il bit R è azzerato ad ogni clock interrupt.

\***current virtual time**=tempo di CPU effettivamente usata dal processo dal momento del suo avvio.

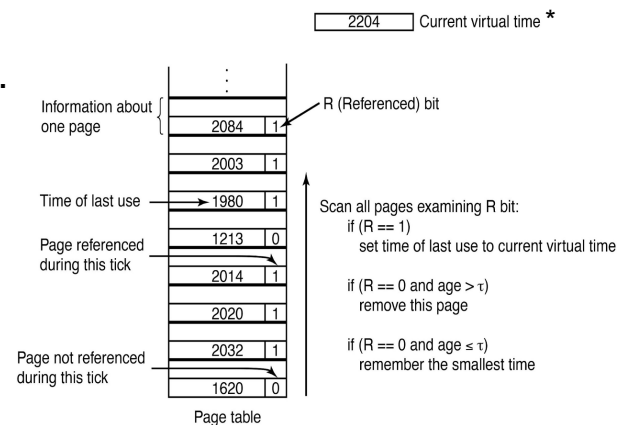


## Algoritmo working set

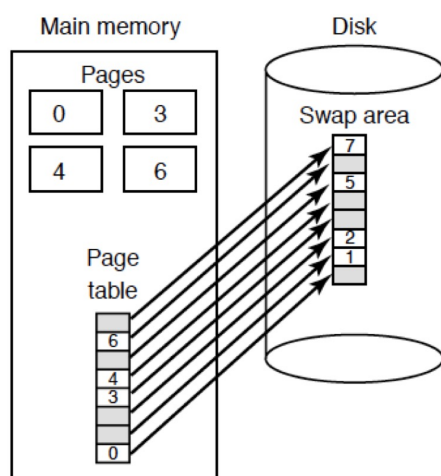
- Ad un page fault si scansiona la tabella delle pagine:
  - se  $R == 1$  il *current virtual time* è scritto nell'*istante di ultimo utilizzo* della PTE della pagina;
  - se  $R == 0$  si calcola l'*età della pagina*\*:
    - età >  $\tau \rightarrow$  **sostituisci**;
    - età  $\leq \tau \rightarrow$  la pagina è risparmiata.

l'algoritmo sostituisce la pagina con  $R==0$   
ed età maggiore di  $\tau$

\*età della pagina = (current virtual time) –  
(istante di ultimo utilizzo)

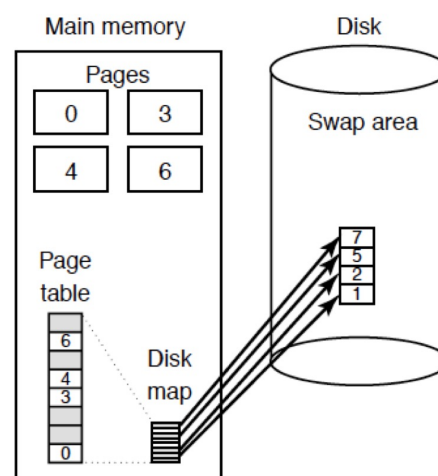


## Area di scambio (par. 3.6.5)



(a)

(a) Paginazione con area di scambio statica.



(b)

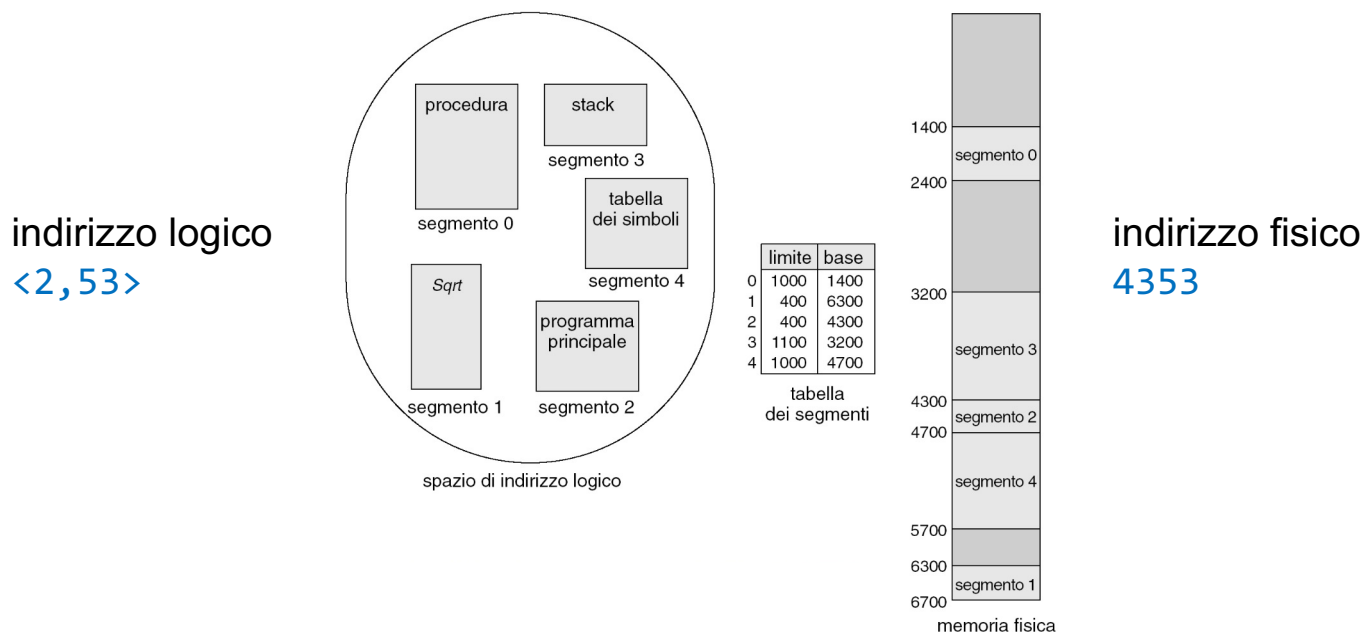
(b) Salvataggio dinamico delle pagine

# NOTE CONCLUSIVE

## Segmentazione: cenni (par. 3.7 fino a 3.7.1 escluso)

- Approccio alternativo alla paginazione per suddividere lo spazio degli indirizzi;
- Un programma è visto come un **insieme di segmenti**; un **segmento** è un'unità logica come:
  - *un programma principale;*
  - *funzioni;*
  - *variabili locali, variabili globali;*
  - *pile;*
  - *tabella dei simboli;*
  - *vettori;*
  - ...

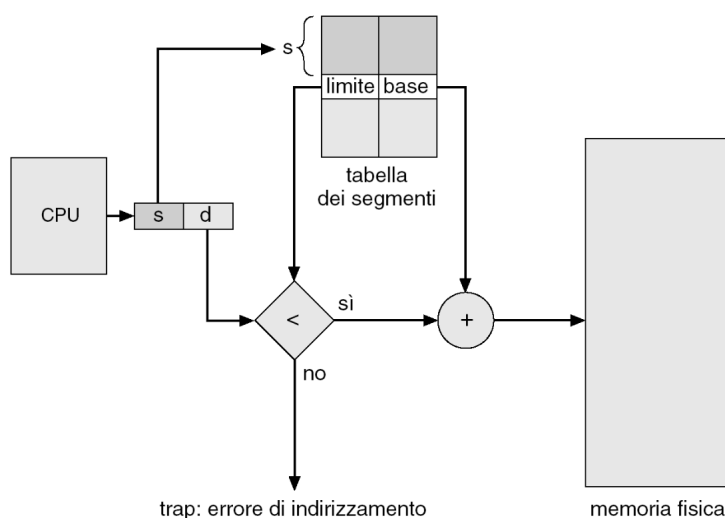
## Segmentazione: cenni (par. 3.7 fino a 3.7.1 escluso)



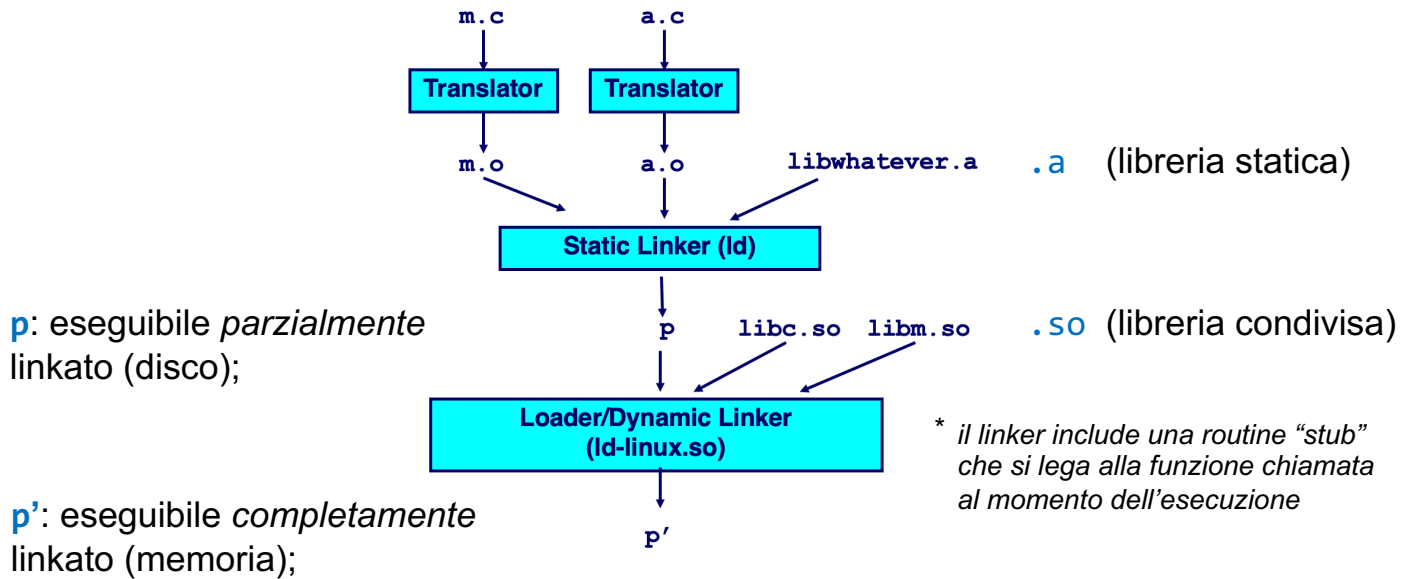
## Segmentazione: cenni (par. 3.7 fino a 3.7.1 escluso)

Dato un **indirizzo logico**  $\langle s, d \rangle$  (numero segmento, scostamento):

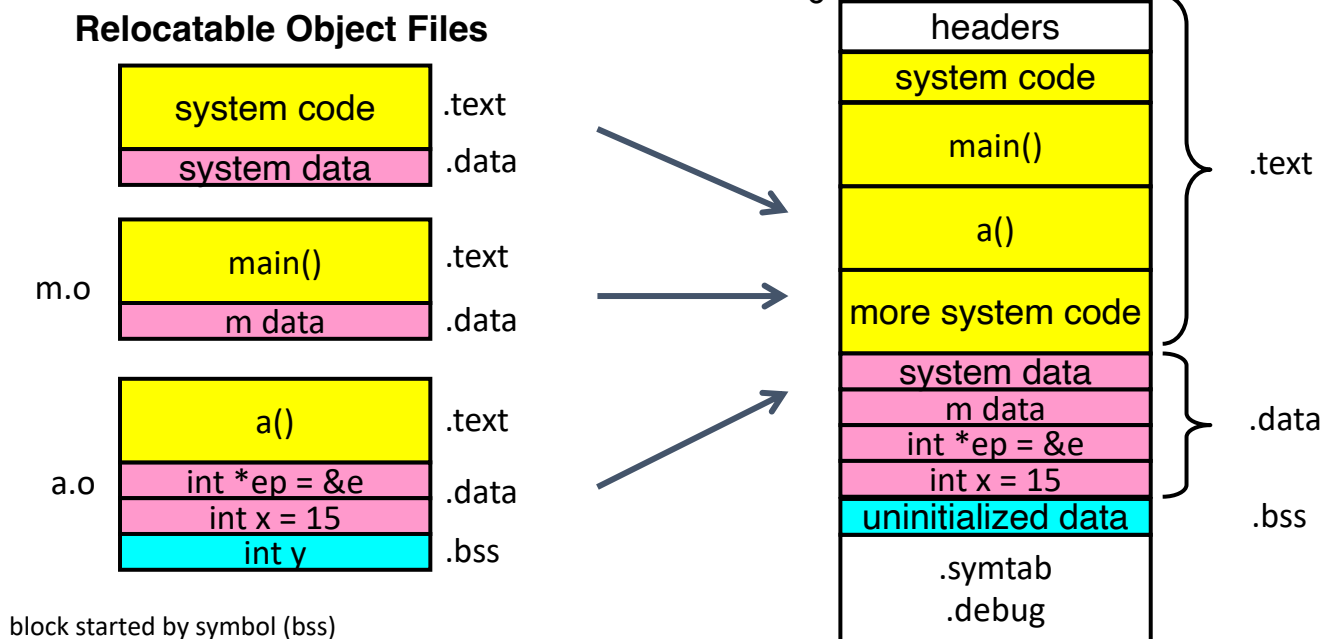
- si usa  $s$  come indice della tabella per ricavare l'indirizzo **base**;
- si verifica che  $d < \text{limite}$ :
  - **vero**: indirizzo fisico = **base** +  $d$ ;
  - **falso**: eccezione.



## Sidenote: file eseguibile



## Sidenote: file eseguibile

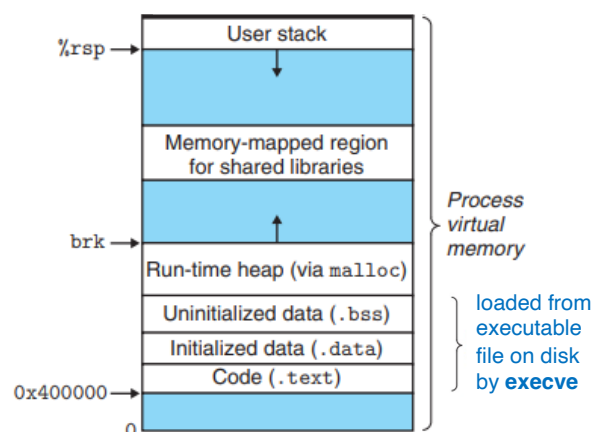
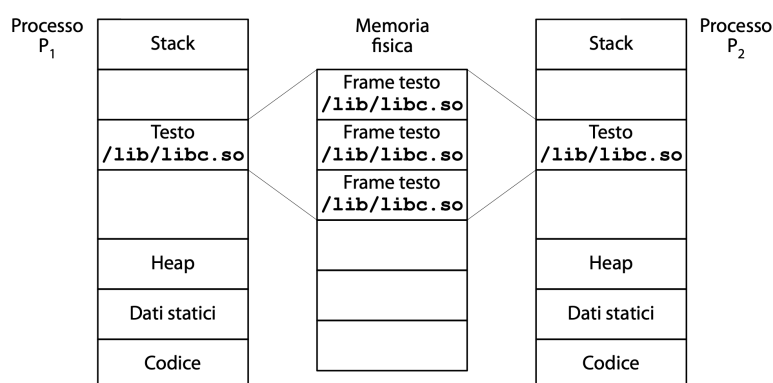




# Linux address space

- Visualizzazione memory map di un processo: `pmap -x -p PID`

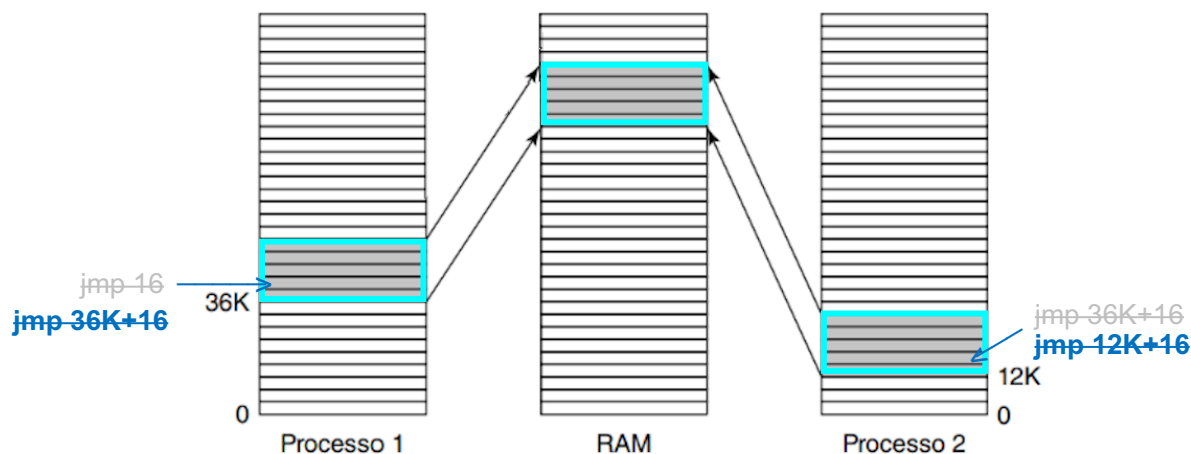
shared library (libreria condivisa)\*



\* Cap. 3 par. 3.5.7

## Librerie condivise (Cap. 3 par 3.5.7)

- Soluzione: **indirizzi relativi**. Per es., `jmp` in avanti (indietro) di  $n$  byte invece di `jmp` ad un certo indirizzo → codice *indipendente* dalla posizione.

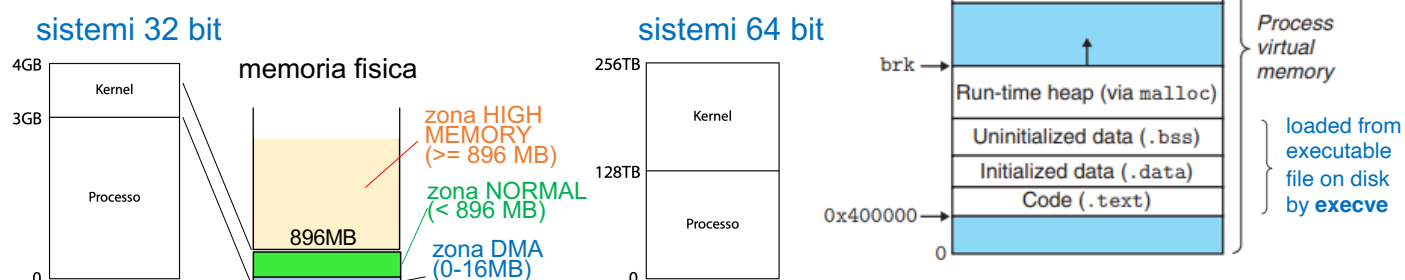


**Figura 3.26** Una libreria condivisa usata da due processi.

# Linux address space

- Lo spazio degli indirizzi è diviso in due *spezzoni*:
  - kernel** (mai sostituito);
  - processo** (sostituito ad ogni cambio di contesto).

## Suddivisione user-kernel

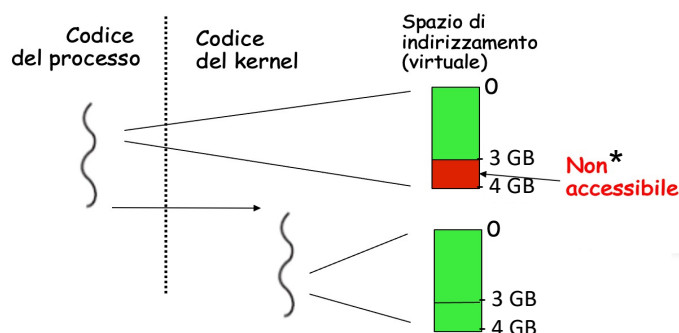


# Linux address space

- Nei sistemi a **32 bit** l'*ultimo* GB è **riservato** per il kernel:
  - il "grosso" (~ 896MB) degli indirizzi dell'ultimo GB sono mappati **linearmente** con la memoria fisica:

$$\text{ind.fisico} = \text{ind.lineare} - 0xC0000000$$

tempo

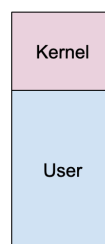


\* più che "non accessibile", con la **Kernel Page Table Isolation (KPTI)** **abilitata**, il kernel gestisce **due** set di tabelle delle pagine: il primo, **completo** (kernel); il secondo (utente) mappa solo uno *stub* minimale (entrata-uscita dal kernel).

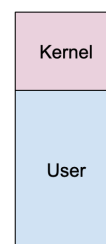
\*\* [https://it.wikipedia.org/wiki/Meltdown\\_\(vulnerabilità\\_di\\_sicurezza\)](https://it.wikipedia.org/wiki/Meltdown_(vulnerabilità_di_sicurezza))

senza KPTI

con KPTI



Before KPTI



After KPTI:  
kernel address space



After KPTI:  
process' address space